Data Link Layer

Framing • Flow & Error Control • Error Detection & Correction • Types of Errors

---

Summarised Study Notes — covers Lecture 13 (Framing, Flow & Error Control) and Lecture 14 (Types of Errors: LRC, Hamming Code, CRC, Checksum). Each topic includes a definition, advantages/disadvantages, and step-by-step numerical solving methods with examples.

1. Introduction to the Data Link Layer

The Data Link Control (DLC) deals with the procedures used for communication between two adjacent nodes — node-to-node communication — regardless of whether the underlying link is dedicated or broadcast. The two main responsibilities of the data link layer are **framing** and **flow & error control** (collectively called *data link control*).

2. Framing

Framing is the process by which the data link layer packs bits into distinguishable units called **frames**, so that each frame can be separated from the next. This is similar to how a postal system separates one letter from another by placing it inside an envelope — the envelope acts as a delimiter. In networking, framing adds a sender address and destination address to a message so it can travel from source to destination; although an entire message could be sent as a single frame, this is normally avoided because very large frames make flow and error control inefficient — a single-bit error in one giant frame would force retransmission of the whole frame, whereas dividing the message into smaller frames confines a single-bit error to just that small frame.

2.1 Frame Size — Two Approaches

Frames can be of **fixed size** (no delimiters needed — size itself acts as the boundary) or **variable size** (needs explicit delimiters). Variable-size framing uses two approaches:

- **Character-oriented (byte-oriented) approach** — data is 8-bit characters (e.g. ASCII).
- **Bit-oriented approach** — data is treated as a raw sequence of bits.

2.2 Character-Oriented Framing

In character-oriented framing, the header (source/destination address, control info) and trailer (error-detection redundant bits) are multiples of 8 bits, same as the data. An 8-bit **FLAG** (a special reserved character, e.g. 0x7E) is placed at the beginning and end of every frame to mark its boundaries.

[Image not found: char_frame.png]

Problem & Solution — Byte Stuffing

If the FLAG's bit pattern happens to occur naturally inside the data (e.g. while sending an image, audio or video), the receiver will mistakenly think the frame has ended there. The fix is **byte stuffing**

(character stuffing): the sender inserts a special 1-byte **ESC (escape character)** immediately before every accidental FLAG or ESC pattern found in the data. The receiver removes every ESC it finds and treats the very next byte as ordinary data rather than a delimiter.

Byte stuffing is the process of adding one extra byte whenever a FLAG or ESC pattern appears in the text.

2.3 Bit-Oriented Framing

Here the data section is a pure sequence of bits (text, graphic, audio, video, etc.). Most protocols use the special 8-bit pattern **01111110** as the FLAG to mark the beginning and end of a frame.

[Image not found: bit_frame.png]

Problem & Solution — Bit Stuffing

Bit stuffing is the process of adding one extra 0 whenever five consecutive 1s follow a 0 in the data, so the receiver never mistakes a data pattern for the flag 01111110. On reception, the receiver removes any 0 that follows five consecutive 1s (un-stuffing) to recover the original data.

2.4 Advantages & Disadvantages of Framing

Advantages	Disadvantages
• Isolates errors — a corrupted bit affects only one small frame, not the whole message. • Enables efficient flow & error control on manageable-sized units. • Allows multiplexing/interleaving of frames from different sources on a shared link. • Sender/destination addressing lets multiple nodes share one broadcast link.	• Extra overhead per frame (flag, header, trailer bits reduce effective throughput). • Stuffing (byte/bit) adds further overhead and processing delay. • Too many small frames increases the overhead-to-data ratio and reduces efficiency. • Synchronization/boundary errors can misalign an entire stream if a flag is lost.

3. Flow Control

Flow control refers to a set of procedures that restrict the amount of data the sender can send before it must wait for an acknowledgment from the receiver. It exists because the sender and receiver are usually different devices with different speeds of processing and storing data — without flow control, a fast sender could overwhelm a slow receiver's buffer, causing data loss.

[Image not found: flow_control.png]

3.1 Role of Buffers

A common solution is to use two **buffers** — one at the sending data-link layer and one at the receiving data-link layer. A buffer is a block of memory that temporarily holds frames. Flow-control signals travel from the receiver (consumer) back to the sender (producer): when the receiver's buffer has space it signals READY; when the buffer becomes full it signals STOP so the sender pauses transmission until space is freed.

3.2 Advantages & Disadvantages of Flow Control

Advantages	Disadvantages
• Prevents receiver buffer overflow and consequent data loss. • Improves overall network reliability and throughput matching. • Allows devices of different speeds to communicate safely.	• Adds control overhead (feedback signals consume bandwidth). • Introduces additional delay while waiting for READY/STOP signals. • Simple stop-based schemes (e.g. Stop-and-Wait) under-utilise link capacity.

4. Error Control

Because the physical layer is not fully reliable, error control is implemented at the data-link layer to prevent a receiving node from delivering a corrupted packet up to its network layer. Error control here is normally simple: a **CRC (redundancy)** is added to the frame by the sender and re-checked by the receiver. Error control in the data-link layer is fundamentally based on **ARQ (Automatic Repeat reQuest)** — the retransmission of data whenever an error is detected.

4.1 Two Methods of Error Control

1. **Detection only (used in wired LANs, e.g. Ethernet):** if the received frame is corrupted, it is silently discarded; if correct, the packet is passed to the network layer. No acknowledgment is sent.
2. **Detection + Acknowledgment (used in reliable data-link protocols):** if the frame is corrupted it is silently discarded (no ACK sent, prompting the sender to eventually retransmit); if correct, an ACK is returned to the sender confirming both error-free delivery and freeing send-buffer space (flow control).

4.2 Combination of Flow and Error Control

Flow and error control are usually combined: the same acknowledgment used for flow control also confirms error-free arrival. The lack of an acknowledgment implies a problem with the sent frame. A frame that carries only an acknowledgment (and no user data) is called an **ACK**, distinguishing it from a normal data frame.

4.3 Advantages & Disadvantages of Error Control

Advantages	Disadvantages
• Guarantees (with ACK) that only correct data reaches upper layers. • ARQ-based retransmission recovers automatically from transient link noise. • Combining flow + error control (ACK) reduces protocol overhead.	• Retransmissions add latency, especially over long/high-latency links. • Detection-only method (no ACK) silently drops corrupted frames — no recovery at this layer. • CRC computation adds processing overhead at both sender and receiver.

5. Error Detection and Correction — Introduction

Data can be corrupted during transmission because of interference that changes the shape of the electrical/optical/radio signal. Some applications merely need to know an error occurred (detection); others need to actually fix it (correction). To detect or correct errors we must send extra **redundant bits** along with the data.

5.1 Types of Errors

Single-bit error: only 1 bit of a given data unit (byte, character, or packet) is changed from 1 to 0 or 0 to 1. **Burst error:** 2 or more bits in the data unit have changed — the corrupted bits need not be consecutive, but the burst length is measured from the first to the last corrupted bit, inclusive.

[Image not found: error_types.png]

5.2 Redundancy & Encoder-Decoder Structure

The core idea of all error-control coding: the sender's **encoder** runs the message through a **generator** that appends redundancy, producing "message + redundancy" which is transmitted over an unreliable channel. The receiver's **decoder** runs the received information through a **checker**; based on the result the message is judged correct or discarded (or, in error-correcting codes, automatically corrected).

5.3 Detection vs Correction

Aspect	Error Detection	Error Correction
Goal	Only know that an error occurred	Identify & fix the exact corrupted bit(s)
Redundancy needed	Less	More
Receiver action on error	Discard frame / request retransmission	Correct data locally, no retransmission needed
Overhead	Lower	Higher

6. Block Coding

In block coding, the message is divided into blocks, each of **k** bits, called **datawords**. To each block, **r** redundant bits are added, making the total length **n = k + r**. The resulting n-bit blocks are called **codewords**. Out of 2■ possible codewords, only 2^k are used (valid); the rest are invalid and signal an error if received.

Example 10.1 (4B/5B coding): here k = 4 and n = 5, so there are 2■ = 16 datawords and 2■ = 32 codewords. Only 16 of the 32 codewords are used for message transfer; the rest are reserved for other purposes or left unused.

6.1 Worked Example — Error Detection (k=2, n=3)

Dataword	Codeword
00	000
01	011
10	101

Dataword	Codeword
11	110

Step-by-step interpretation (sender encodes dataword 01 → sends codeword 011):

1. **Case 1 — No error:** Receiver gets 011. It matches a valid codeword in the table, so dataword 01 is correctly extracted.
2. **Case 2 — Detected error:** Codeword corrupted in transit to 111. This is NOT in the valid codeword list, so it is discarded — error detected.
3. **Case 3 — Undetected error:** Codeword corrupted to 000. This IS a valid codeword (for dataword 00), so the receiver wrongly accepts dataword 00. Two bit-flips happened to land on another valid codeword, making the error undetectable.

An error-detecting code can only detect the types of errors it is specifically designed for; other error patterns may remain undetected.

6.2 Worked Example — Error Correction (k=2, n=5)

Dataword	Codeword
00	00000
01	01011
10	10101
11	11110

Sender sends dataword 01 as codeword 01011; it is corrupted to 01001. Correction steps:

1. Receiver checks 01001 against the codeword table — it is not a valid entry, so an error is confirmed.
2. Assuming only 1 bit is corrupted, compare 01001 with each valid codeword and count differing bits.
3. vs. 00000 → 2 bits differ → rejected as the original.
4. vs. 01011 → only 1 bit differs (position 4) → accepted as the most likely original.
5. vs. 10101 and 11110 → both differ by more than 1 bit → rejected.
6. Receiver corrects 01001 → 01011, then looks it up to recover dataword **01**.

7. Linear Block Codes (LRC)

Almost all block codes used today belong to a subset called **linear block codes**. A code is linear if the XOR (modulo-2 addition) of any two valid codewords produces another valid codeword. In a linear block code, the **minimum Hamming distance (d_{min})** equals the smallest number of 1s found among all non-zero valid codewords — this single number determines the code's detection and correction power.

7.1 Simple Parity-Check Code

A simple parity-check code is a single-bit error-detecting linear block code where n = k + 1 and d_{min} = 2. One redundant parity bit r■ is computed as the modulo-2 sum of all data bits: r■ = a■ + a■ + a■ + a■. At the receiver, a **syndrome** is calculated the same way including the received parity bit; s■ = b■ + b■ + b■ + b■ + q■. If s■ = 0 the frame is accepted; if s■ = 1 it is discarded.

[Image not found: parity_check.png]

A simple parity-check code can detect only an odd number of bit errors (1, 3, 5, ...); an even number of errors cancels out and goes undetected.

7.2 Two-Dimensional Parity Check

To catch more error patterns, data is arranged as rows, and a parity bit is computed for every row AND for every column, forming an extra row and column of parity bits.

[Image not found: two_d_parity.png]

1. A single-bit error flips one row parity AND one column parity → detected and even locatable.
2. A two-bit error (same row or column) still flips at least two parities → detected.
3. Three-bit errors typically flip 3-4 parities → still detected.
4. A specific pattern of exactly four errors at the four corners of a rectangle can cancel all parities and go **undetected**.

Two-dimensional parity check can reliably detect up to 3-bit (burst) errors.

8. Hamming Code

The Hamming code is a linear block error-**correcting** code. For a 4-bit dataword (a■a■a■a■), three redundant parity bits (r■ r■ r■) are computed using overlapping modulo-2 sums, producing the well-known **Hamming code C(7,4)** with n = 7, k = 4, d_{min} = 3. It can detect up to 2-bit errors and correct 1-bit errors.

Parity-bit generation equations (modulo-2 addition):

- r0 = a2 + a1 + a0
- r1 = a3 + a2 + a1
- r2 = a1 + a0 + a3

[Image not found: hamming_structure.png]

Syndrome equations calculated by the receiver's checker:

- s0 = b2 + b1 + b0 + q0
- s1 = b3 + b2 + b1 + q1
- s2 = b1 + b0 + b3 + q2

Syndrome-to-error lookup table used by the correction logic:

Syndrome (s2 s1 s0)	000	001	010	011	100	101	110	111
Bit in error	None	q0	q1	b2	q2	b0	b3	b1

Hamming code C(7,4) can detect up to 2-bit errors ($d_{min} - 1$) and correct up to 1-bit errors (($d_{min} - 1$)/2).

8.1 Worked Example (Concept Check) — Dataword 0010, C(7,4), Even Parity

Given: dataword a3a2a1a0 = 0010 (a3=0, a2=0, a1=1, a0=0). Generate the codeword using even parity, then assume the q0 bit is received with an error, and correct it.

Step 1 — Compute the redundancy bits at the sender:

1. r0 = a2 + a1 + a0 = 0 + 1 + 0 = **1**
2. r1 = a3 + a2 + a1 = 0 + 0 + 1 = **1**
3. r2 = a1 + a0 + a3 = 1 + 0 + 0 = **1**
4. Codeword transmitted = a3 a2 a1 a0 r2 r1 r0 = 0 0 1 0 1 1 1 → **0010111**

Step 2 — Introduce the error (q0 flips):

1. Sent codeword: 0010111
2. q0 (rightmost bit, r0 position) flips from 1 → 0
3. Received codeword b3 b2 b1 b0 q2 q1 q0 = **0010110**

Step 3 — Receiver computes the syndrome:

1. s0 = b2 + b1 + b0 + q0 = 0 + 1 + 0 + 0 = **0**
2. s1 = b3 + b2 + b1 + q1 = 0 + 0 + 1 + 1 = **0**
3. s2 = b1 + b0 + b3 + q2 = 1 + 0 + 0 + 1 = **0**
4. Wait — check directly against original bits: with q0 flipped, only s0 (which uses q0) becomes non-zero.
5. Recomputing correctly: s0 = b2+b1+b0+q0 = 0+1+0+0 = 1 (since correct q0 should be 1, now 0 → parity fails)
6. s1 = b3+b2+b1+q1 = 0+0+1+1 = 0 (unaffected, still even)
7. s2 = b1+b0+b3+q2 = 1+0+0+1 = 0 (unaffected, still even)
8. Syndrome = s2 s1 s0 = **001**

Step 4 — Look up the syndrome table:

1. Syndrome 001 → corresponds to an error in bit **q0** (matches the table row above).
2. Correction logic flips q0 back: 0 → 1.
3. Corrected codeword = 0010111, which matches the originally sent codeword.
4. Dataword extracted from a3a2a1a0 = **0010** ✓ — successfully corrected without retransmission.

8.2 Burst Error Correction Using Hamming Code

A single Hamming codeword can correct only 1 bit, so it cannot directly fix a burst error spanning several consecutive bits within one codeword. The trick is to **interleave**: several codewords are transmitted column-by-column (transposed) instead of row-by-row. A burst error in the transmitted bit-stream then lands on a **different bit position of a different codeword** each time, so each affected codeword sees only a single-bit error — which the Hamming code can correct independently.

9. Cyclic Codes and CRC (Cyclic Redundancy Check)

Cyclic codes are a special subset of linear block codes with one extra property: if a codeword is cyclically shifted (rotated), the result is still a valid codeword. The most widely used cyclic code in networking is the **CRC (Cyclic Redundancy Check)**. Instead of parity equations, CRC treats bit strings as polynomials and uses **binary (modulo-2) division**: the sender appends r zero bits to the dataword and divides the result by a fixed **generator/divisor** (of r+1 bits); the **remainder** of that division becomes the CRC redundancy bits appended to the actual codeword.

[Image not found: crc_structure.png]

9.1 Advantages & Disadvantages of CRC

Advantages	Disadvantages
• Very strong at detecting burst errors (a well-chosen generator catches all bursts shorter than r bits). • Detects all single-bit and all odd-numbered bit errors. • Efficient hardware/software implementation using shift registers or polynomial division. • Widely standardized (CRC-8, CRC-16, CRC-32) and used in Ethernet, ZIP, etc.	• Only detects errors — cannot correct them by itself; needs retransmission (ARQ). • Choice of generator polynomial is critical; a poor choice weakens detection. • Adds fixed overhead of r redundant bits per frame. • Cannot detect certain rare error patterns that are exact multiples of the generator polynomial.

9.2 Worked Example — CRC Division at the Encoder

Given: Dataword = 1001, Divisor (generator) = 1011 (4 bits, so r = 3 redundant bits are needed).

Step-by-step binary (XOR / modulo-2) division:

1. Augment the dataword with r = 3 zero bits: 1001 **000** (the "dividend").
2. Divide 1001000 by the divisor 1011 using XOR at each step (not normal subtraction).
3. Bring down the leading 4 bits (1001); since the leftmost bit is 1, XOR with the divisor 1011 → result 0010.
4. Bring down the next bit (0) → 0100; leftmost bit is 0, so XOR with 0000 → 0100 unchanged.
5. Bring down the next bit (0) → 1000; leftmost bit is 1, so XOR with 1011 → 0011.
6. Bring down the last bit (0) → 0110; leftmost bit is 0, so XOR with 0000 → remainder stops here.
7. Final **remainder = 110** (these are the r0 r1 r2 redundancy/CRC bits).
8. Transmitted codeword = Dataword + Remainder = 1001 + 110 = **1001110**

9.3 Worked Example — CRC Check at the Decoder

Case A — No error: Receiver gets codeword 1001110.

1. Divide the received codeword 1001110 by the same divisor 1011.
2. Perform the modulo-2 division exactly as at the sender.
3. Resulting syndrome (remainder) = **000**.
4. Syndrome is all-zero → frame accepted → dataword 1001 extracted.

Case B — Error introduced: Receiver gets codeword 1000110 (one bit flipped).

1. Divide 1000110 by the divisor 1011 in the same way.
2. Resulting syndrome (remainder) = **011** (non-zero).
3. Since the syndrome is not all-zero, an error is detected $\rightarrow$ frame is discarded.
4. Data-link layer requests / awaits retransmission (ARQ) since CRC only detects, it does not correct.

10. Checksum

The checksum is another error-detection technique, commonly used at higher layers (e.g. in IP, UDP, TCP headers) though conceptually part of the same redundancy-based error-control family covered alongside LRC and CRC. The data unit is divided into equal-size segments (typically 16-bit words); these segments are added together using one's complement arithmetic, and the complement of the final sum is sent as the **checksum** field along with the data. The receiver performs the same addition (including the received checksum) — if the complement of the sum is all zeros, the data is accepted as error-free; otherwise, it is discarded.

10.1 Step-by-Step Checksum Procedure (Sender)

1. Divide the data into k segments, each of a fixed number of bits (e.g. 16 bits).
2. Add all k segments together using one's complement addition (any end-around carry is added back to the sum).
3. Complement (invert every bit of) the final sum to obtain the checksum.
4. Append the checksum segment to the original data and transmit.

10.2 Step-by-Step Checksum Procedure (Receiver)

1. Divide the received unit into the same k+1 segments (data segments + checksum segment).
2. Add all segments together (including the received checksum) using one's complement addition.
3. Complement the resulting sum.
4. If the result is all 0s → data accepted as correct; if any bit is 1 → an error is detected and the data is discarded.

10.3 Advantages & Disadvantages of Checksum

Advantages	Disadvantages
• Simple and fast to compute in software, low processing overhead. • Small, fixed-size redundancy regardless of data length. • Effective at catching common transmission errors (e.g. dropped/duplicated bytes).	• Weaker error-detection strength than CRC — can miss certain error combinations that cancel out in the sum. • Cannot correct errors — detection only. • Less effective against burst errors compared to CRC.

11. Quick Comparison — Error Control Techniques

Technique	Error-Handling Capability	Redundancy Overhead	Computational Cost
Simple Parity Check	Detects odd number of bit errors only	1 bit	Very Low
Two-Dimensional Parity	Detects up to 3-bit errors reliably	n+1 (row+col) bits	Low
Hamming Code C(7,4)	Detects 2-bit, corrects 1-bit errors	3 bits (for k=4)	Medium

Technique	Error-Handling Capability	Redundancy Overhead	Computational Cost
CRC	Detects virtually all burst & odd-bit errors	r bits (generator-dependent)	Medium-High
Checksum	Detects most errors; weaker than CRC	Fixed segment (e.g. 16 bits)	Low-Medium

---

End of Notes